PriceIQ

AI-Powered Electronics Price Comparison for Indian Consumers

Test Plan and Test Cases

Document Version 1.0

Project Type:	BTech Final Year Project
Department:	Computer Science and Engineering
Academic Year:	2025–2026
Prepared By:	Adrij Bhadra
Date:	April 2026
Document Status:	Final

Covers: Unit Tests, Integration Tests, System / End-to-End Tests, Performance Tests, Security Tests, and Traceability Matrix.

Contents

1 Introduction **3**

1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Testing Approach and Status . . . 3
1.4 References . . . 4

2 Test Strategy **5**

2.1 Testing Types . . . 5
2.2 Entry and Exit Criteria . . . 5
2.3 Defect Classification . . . 6

3 Test Environment **7**

3.1 Hardware . . . 7
3.2 Software . . . 7
3.3 Environment Variables for Testing . . . 7

4 Unit Test Cases **9**

4.1 normalizeText() . . . 9
4.2 parseQuery() . . . 10
4.3 scoreCandidate() — Hard Rejects . . . 11
4.4 scoreCandidate() — Soft Scores . . . 12

5 Integration Test Cases **14**

5.1 POST /api/search/live . . . 14
5.2 GET /api/search . . . 15
5.3 POST /api/ai/analyze . . . 16
5.4 POST /api/scrape . . . 16
5.5 GET /api/price-history/[id] . . . 17

6 System / End-to-End Test Cases **18**

7 Performance Test Cases **21**

8 Security Test Cases **23**

9 Requirements Traceability Matrix **25**

10 Test Summary **27**

10.1 Test Case Count by Category 27

10.2 Known Limitations and Risks 27

10.3 Recommended Test Execution Order 28

Chapter 1

Introduction

1.1 Purpose

This Test Plan defines the testing strategy, test environment, and test cases for the PriceIQ system. The document covers **56 test cases** across five testing categories: Unit, Integration, System / End-to-End, Performance, and Security. The primary goal is to verify that each functional requirement (FR) defined in the SRS is correctly implemented and that the system meets its non-functional requirements.

1.2 Scope

In scope:

- Unit tests for the `matcher.ts` scoring engine (pure functions).
- Unit tests for utility functions in scraper modules.
- Integration tests for all REST API endpoints.
- System tests for key user workflows (search, compare, AI analysis).
- Performance tests for response latency and load.
- Security tests for the authenticated scrape endpoint and input handling.

Out of scope:

- End-to-end browser tests requiring Playwright against a live deployment.
- Load tests simulating more than 10 concurrent users.
- Testing of the Groq API itself (third-party service).

1.3 Testing Approach and Status

The PriceIQ codebase does not yet include an automated test suite. This document specifies the full set of planned test cases, written from code analysis. All cases are marked **PLANNED** (Planned); they would be executed using **Jest** (unit/integration) and **Playwright Test** (system/E2E) once a test harness is configured. The product matching engine (`matcher.ts`) uses pure functions with no I/O and is the highest priority for automated testing.

TypeScript type correctness is enforced via the Next.js build pipeline (`next build`) since the project does not have a standalone `typescript` dev dependency; the build is confirmed green in the project history.

1.4 References

- PriceIQ Software Requirements Specification v1.0 (`srs.tex`)
- PriceIQ System Design Document v1.0 (`system_design.tex`)
- Source files: `src/lib/scrapers/matcher.ts`, `src/lib/scrapers/live.ts`, `src/app/api/sea`

Chapter 2

Test Strategy

2.1 Testing Types

Type	Tool	Description
Unit Testing	Jest + ts-jest	Tests individual pure functions in `matcher.ts` and utility modules in isolation. No network or database access.
Integration Testing	Jest + Supertest	Tests API route handlers against a test PostgreSQL database using real HTTP calls. Scrapers are mocked to return deterministic fixtures.
System / E2E Testing	Manual / Playwright Test	Tests complete user workflows in a running application instance. Validates UI rendering, navigation, and data accuracy end-to-end.
Performance Testing	Manual + `k6`	Measures response latency and throughput for critical paths under realistic load conditions.
Security Testing	Manual + OWASP ZAP	Verifies authentication on protected endpoints, input validation, and absence of sensitive data leakage.

2.2 Entry and Exit Criteria

Phase	Entry Criteria	Exit Criteria
Unit Testing	`matcher.ts` source compiles with zero TypeScript errors.	All 27 unit test cases pass with zero failures.
Integration Testing	Application starts successfully against a test database.	All 17 integration test cases pass; no 5xx responses from valid inputs.
System Testing	Application deployed locally with Docker PostgreSQL running.	All 8 critical system test cases pass.
Performance Testing	System tests passed; performance baselines established.	All latency metrics within the NFR thresholds defined in the SRS.
Security Testing	Application running with production-equivalent environment variables.	All 4 security test cases pass.

2.3 Defect Classification

Severity	Label	Definition
1	Critical	Data corruption, wrong product stored in DB, security bypass, system crash on valid input. Must fix before release.
2	High	Feature does not work as specified (wrong score, missed hard-reject, broken API route). Must fix before evaluation.
3	Medium	UI glitch, minor performance issue, non-critical API field missing.
4	Low	Cosmetic, typo, log verbosity. Fix in next iteration.

Chapter 3

Test Environment

3.1 Hardware

- Development machine: Windows 11 Home Single Language 10.0.26200
- RAM: 8 GB minimum (Next.js + PostgreSQL + Playwright Chromium)
- Internet: Required for live scrape tests (VS GraphQL, Flipkart)

3.2 Software

Component	Version	Purpose
Node.js	20+	Runtime for Next.js and Jest
Docker Desktop	24+	PostgreSQL container
PostgreSQL	16 (Docker image)	Test database (separate from dev)
Next.js	16.2.4	Application under test
Jest	29+ (to install)	Unit and integration test runner
ts-jest	29+ (to install)	TypeScript transform for Jest
Playwright Test	1.59.1	System / E2E test runner

3.3 Environment Variables for Testing

```
DATABASE_URL=postgresql://aggregator:aggregator@localhost:54321/aggregator_test
GROQ_API_KEY=<mock-key-for-integration-tests>
```

```
SCRAPE_SECRET=test-secret-1234
NODE_ENV=test
```

Chapter 4

Unit Test Cases

Unit tests cover pure functions in `src/lib/scrapers/matcher.ts`. No network calls or database access are required.

4.1 normalizeText()

TC-ID	Test Name	Input	Expected Output	Status
TC-U001	Basic lowercase conversion	`"Apple iPhone 15"`	`"apple iphone 15"`	PLANNED
TC-U002	Plus sign to “plus”	`"Galaxy S24+"`	`"galaxy s24 plus"`	PLANNED
TC-U003	Model code join (letter-digit dash)	`"Sony WH-1000XM5"`	`"sony wh1000xm5"`	PLANNED
TC-U004	Model code join does not affect digit-digit	`"Samsung 55-inch TV"`	`"samsung 55 inch tv"`	PLANNED
TC-U005	Remaining dashes become spaces	`"On-Ear Headphones"`	`"on ear headphones"`	PLANNED
TC-U006	Special characters removed	`"iPhone 15 (128GB)"`	`"iphone 15 128gb"`	PLANNED
TC-U007	Multiple spaces collapsed	`"iPhone 15 Pro"`	`"iphone 15 pro"`	PLANNED

continued on next page...

TC-ID	Test Name	Input	Expected Output	Status
TC-U008	Em-dash and underscores removed	`"WF--1000XM4_v2"`	`"wf 1000xm4 v2"`	PLANNED

4.2 parseQuery()

TC-ID	Test Name	Input	Expected Output	Status
TC-U009	Brand extraction — Apple	`parseQuery("Apple iPhone 15 128GB",...)`	`brand: "apple"`	PLANNED
TC-U010	Brand extraction — Sony	`parseQuery("Sony WH-1000XM5",...)`	`brand: "sony"`	PLANNED
TC-U011	Storage token 128GB	`parseQuery("iPhone 15 128GB",...)`	`storageToken: "128gb"`	PLANNED
TC-U012	Storage token 256GB	`parseQuery("Galaxy S25 256GB",...)`	`storageToken: "256gb"`	PLANNED
TC-U013	Storage token 1TB	`parseQuery("MacBook Pro 1TB",...)`	`storageToken: "1tb"`	PLANNED
TC-U014	Variant — Ultra	`parseQuery("Galaxy S25 Ultra",...)`	`variantToken: "ultra"`	PLANNED
TC-U015	Variant — Pro Max before Pro	`parseQuery("iPhone 15 Pro Max",...)`	`variantToken: "pro max"`	PLANNED
TC-U016	Model token extraction	`parseQuery("iPhone 15 128GB",...)`	`modelTokens: ["15"]`	PLANNED
TC-U017	Storage token excluded from model tokens	`parseQuery("iPhone 15 128GB",...)`	`modelTokens does not contain "128gb"`	PLANNED
TC-U018	Model code as single token (WH-1000XM5)	`parseQuery("Sony WH-1000XM5",...)`	`modelTokens: ["wh1000xm5"]`	PLANNED
TC-U019	hasRejectPhrase — renewed query	`parseQuery("Renewed iPhone 14",...)`	`hasRejectPhrase: true`	PLANNED

continued on next page...

TC-ID	Test Name	Input	Expected Output	Status
TC-U020	hasRejectPhrase — normal query	`parseQuery("iPhone 15 128GB",...)`	`hasRejectPhrase: false`	PLANNED

4.3 scoreCandidate() — Hard Rejects

TC-ID	Test Name	Candidate Title	Expected Result	Status
TC-U021	Reject accessory (case)	`"iPhone 15 128GB Case"`	`accepted: false, score: -100`	PLANNED
TC-U022	Reject accessory (charger)	`"20W USB-C Charger for iPhone"`	`accepted: false, score: -100`	PLANNED
TC-U023	Reject accessory (tempered glass)	`"Tempered Glass for Samsung S24"`	`accepted: false, score: -100`	PLANNED
TC-U024	Reject renewed (not requested)	`"Renewed iPhone 15 128GB"`	query hasRejectPhrase=false ⇒ `accepted: false, score: -80`	PLANNED
TC-U025	Allow renewed (query requests it)	`"Renewed iPhone 15 128GB"`	query hasRejectPhrase=true ⇒ `accepted: true`	PLANNED
TC-U026	Reject zero model token match	`"Apple iPhone 14 128GB"`	query modelTokens=[”15”] ⇒ `accepted: false, score: -50`	PLANNED
TC-U027	Reject storage conflict (256GB vs 128GB)	`"Apple iPhone 15 256GB"`	query storageToken=”128gb” ⇒ `accepted: false, score: -60`	PLANNED

continued on next page...

TC-ID	Test Name	Candidate Title	Expected Result	Status
TC-U028	Reject variant conflict (Pro vs Ultra)	`"Samsung Galaxy S25 Pro 256GB"`	query variantToken=”ultra” ⇒ `accepted: false, score: -60`	PLANNED

4.4 scoreCandidate() — Soft Scores

TC-ID	Test Name	Candidate Title	Expected Result	Status
TC-U029	Brand match earns +30	`"Apple iPhone 15 128GB"`	brand ”apple” present ⇒ score includes +30	PLANNED
TC-U030	Missing brand deducts -30	`"Generic Phone 15 128GB"`	query brand=”apple” ⇒ score includes -30	PLANNED
TC-U031	All model tokens matched — +40	`"iPhone 15 128GB"`	modelTokens=[”15”] fully matched ⇒ +40	PLANNED
TC-U032	Partial model token match (half)	`"Galaxy S24 256GB"`	modelTokens=[”s24” (1/2 matched) ⇒ +20	PLANNED
TC-U033	Storage match +20	`"iPhone 15 128GB"`	storageToken=”128g matched ⇒ +20	PLANNED
TC-U034	No storage in title -5	`"iPhone 15 Midnight"`	storageToken=”128g no GB in title ⇒ -5	PLANNED
TC-U035	Variant match +20	`"Galaxy S25 Ultra 256GB"`	variantToken=”ultra matched ⇒ +20	PLANNED
TC-U036	Unexpected variant -15	`"Galaxy S25 Ultra 256GB"`	query no variantToken ⇒ -15	PLANNED
TC-U037	Combo/bundle penalty -30	`"iPhone 15 128GB + AirPods Bundle"`	query no bundle keyword ⇒ -30	PLANNED

continued on next page...

TC-ID	Test Name	Candidate Title	Expected Result	Status
TC-U038	TV title with “4K Ultra HD” — no variant penalty	`"Samsung 55 inch 4K Ultra HD Smart TV"`	query no variantToken ⇒ “ultra hd” stripped before detection; no -15 penalty	**PLANNED**
TC-U039	Full iPhone 15 128GB positive score	`"Apple iPhone 15 128GB Blue"`	query ”Apple iPhone 15 128GB” ⇒ score $\geq$ 80, accepted: true	**PLANNED**
TC-U040	Vague query accepted at lower threshold	`"Sony WH-1000XM5 Headphones"`	query ”Sony headphones”, modelTokens=[] ⇒ threshold=20, accepted: true	**PLANNED**

Chapter 5

Integration Test Cases

Integration tests exercise the Next.js API route handlers against a real (test) PostgreSQL database. External scrapers are mocked to return fixture data.

5.1 POST /api/search/live

TC-ID	Test Name	Input / Precondition	Expected Result	Status
TC-I001	Valid query returns 200 with productId	Body: `{"query":"iPhone 15 128GB"}`	HTTP 200; response body contains `productId` (integer)	PLANNED
TC-I002	Missing query returns 400	Body: {}	HTTP 400; body contains `{"error":"Query is required"}`	PLANNED
TC-I003	Empty query string returns 400	Body: `{"query":""}`	HTTP 400	PLANNED
TC-I004	Product record created in DB	First time querying "iPhone 15 128GB"	`Product` with slug `iphone-15-128gb` created in DB	PLANNED

continued on next page...

TC-ID	Test Name	Input / Precondition	Expected Result	Status
TC-I005	ProductListing created per scraper result	Mock returns VS + FK results	Two `ProductListing` rows with source vijaysales and flipkart	PLANNED
TC-I006	PriceHistory row appended	Fixture product price = 58190	One `PriceHistory` row per listing with correct price	PLANNED
TC-I007	Second call upserts, not duplicates	Same query called twice	DB still has one `Product` row; listing price updated	PLANNED
TC-I008	Response includes AI analysis	Groq mock returns {`summary,recommen`	Response body contains `aiAnalysis.summar` (non-empty string)	PLANNED
TC-I009	AI failure does not block response	Groq mock throws network error	HTTP 200 still returned; `aiAnalysis` may be null	PLANNED

5.2 GET /api/search

TC-ID	Test Name	Query Params	Expected Result	Status
TC-I010	Full-text search by name	`?q=iphone`	Returns products where name contains "iphone" (case-insensitive)	PLANNED
TC-I011	Category filter	`?category=Smartph`	All returned products have category = "Smartphones"	PLANNED

continued on next page...

TC-ID	Test Name	Query Params	Expected Result	Status
TC-I012	Sort by price ascending	`?sort=price_asc`	Products ordered by minimum listing price ascending	PLANNED
TC-I013	Sort by rating descending	`?sort=rating`	Products ordered by average rating descending	PLANNED
TC-I014	No results for unknown query	`?q=xyznoproduct99`	HTTP 200; `data: []`	PLANNED

5.3 POST /api/ai/analyze

TC-ID	Test Name	Input / Precondition	Expected Result	Status
TC-I015	Returns AI analysis for valid product	Existing productId, no cached analysis	HTTP 200; body contains `summary`, `recommendation`, `bestSource`	PLANNED
TC-I016	Cached result returned within 24h	Product has AIAnalysis row $<$ 24h old	HTTP 200; body contains `cached: true`	PLANNED
TC-I017	Refreshes after 24h cache expiry	AIAnalysis row $>$ 24h old	Groq is called; new analysis stored; `cached: false`	PLANNED

5.4 POST /api/scrape

TC-ID	Test Name	Input / Precondition	Expected Result	Status
TC-I018	Missing secret rejected	No `x-scrape-secret` header	HTTP 403 Forbidden	PLANNED
TC-I019	Wrong secret rejected	Header: `x-scrape-secret: wrongvalue`	HTTP 403 Forbidden	PLANNED

TC-ID	Test Name	Input / Precondition	Expected Result	Status
TC-I020	Correct secret triggers scrapers	Header: correct SCRAPE_SECRET; scrapers mocked	HTTP 200; response contains `upserted` count ≥ 1	**PLANNED**

5.5 GET /api/price-history/[id]

TC-ID	Test Name	Input	Expected Result	Status
TC-I021	Returns price history for known product	Valid productId with 3 history rows	HTTP 200; array of 3 objects with `price`, `recordedAt`, `source`	**PLANNED**
TC-I022	Returns empty array for product with no history	Valid productId, no PriceHistory rows	HTTP 200; `data: []`	**PLANNED**
TC-I023	Invalid productId returns 404	Non-existent id (e.g., 999999)	HTTP 404	**PLANNED**

Chapter 6

System / End-to-End Test Cases

System tests validate complete user workflows in a running application. Tests are performed manually unless otherwise noted.

TC-ID	Test Name	Steps	Expected Result	Status
TC-S001	Home page loads correctly	Navigate to `http://localhost:30(`	Hero section visible with search bar; dark theme applied; no console errors	PLANNED
TC-S002	Search for existing product — cache hit	Database seeded; enter "iPhone 15" in search bar; press Enter	Results page shows at least one product card with correct name, price, and platform badge; response within 500 ms	PLANNED
TC-S003	Search for unknown product triggers live search	Database empty; enter "Sony WH-1000XM5" in search bar	LiveSearchTrigger appears with animated platform steps (Amazon, Flipkart, Vijay Sales); after completion, redirects to product page	PLANNED

continued on next page...

TC-ID	Test Name	Steps	Expected Result	Status
TC-S004	Live search animated steps display correctly	While live search is running (observe UI)	Each platform shows: "Searching…" → "Found it!" (or skipped with amber warning); steps do not skip prematurely	**PLANNED**
TC-S005	Product page shows price comparison	Navigate to `/product/{id}` for a product with VS + FK listings	Price Comparison tab shows two listing cards; cheapest marked "Best Deal"; price difference percentage visible on the other card	**PLANNED**
TC-S006	AI Analysis tab renders recommendation	Click "AI Analysis" tab on product page	Loading skeleton visible briefly; then summary, recommendation, and bestSource badge appear; no console errors	**PLANNED**
TC-S007	Price history chart renders	Product page with at least 2 price history rows	Line chart visible on product page below listing cards; axes labelled with dates and INR values	**PLANNED**
TC-S008	Category filter narrows results	Search results page with mixed categories; select "Smartphones" filter	Only Smartphone products shown; count decreases or stays equal	**PLANNED**

continued on next page…

TC-ID	Test Name	Steps	Expected Result	Status
TC-S009	Sort by price ascending	Search results page; select "Price: Low to High" sort	Products displayed in ascending price order; verified by comparing first and last card prices	**PLANNED**
TC-S010	Dashboard KPI cards load	Navigate to `/dashboard` with seeded data	KPI cards show non-zero values for total products, listings, and searches; pie and bar charts render without errors	**PLANNED**
TC-S011	Refresh Prices button re-triggers live scrape	On product page, click "Refresh Prices"	Toast notification "Refreshing…" appears; on completion, toast says success; PriceHistory row count increases by at least 1 in DB	**PLANNED**
TC-S012	Dark mode persists across navigation	Open app in dark mode; navigate from home → search → product → dashboard	Dark theme maintained on all pages; no flash of light theme	**PLANNED**

Chapter 7

Performance Test Cases

Performance tests validate that the system meets the NFRs defined in the SRS.

TC-ID	Test Name	Method / Tool	Acceptance Criterion	Status
TC-P001	Live search total latency (all 3 platforms)	Measure wall-clock time from POST `/api/search/live` to response; repeat 3 times; take median	Median $\leq$ 60 seconds on a 20 Mbps connection	**PLANNED**
TC-P002	Cached database search latency	GET `/api/search?q=iph` against seeded DB; measure response time with browser DevTools Network tab	Response time $\leq$ 500 ms; TTFB $\leq$ 200 ms	**PLANNED**
TC-P003	Product page SSR latency	Navigate to `/product/{id}` (populated product); measure TTFB via DevTools	TTFB $\leq$ 2 seconds	**PLANNED**

continued on next page...

TC-ID	Test Name	Method / Tool	Acceptance Criterion	Status
TC-P004	AI analysis latency (Groq API)	POST `/api/ai/analyze` with a product having 3 listings; measure from request to JSON response	Response time $\leq$ 8 seconds	**PLANNED**
TC-P005	AI analysis cache hit latency	POST `/api/ai/analyze` for a product with cached analysis (< 24h)	Response time $\leq$ 100 ms (DB-only, no Groq call)	**PLANNED**
TC-P006	Vijay Sales scraper latency	Measure `liveVS()` execution time for one product query (3 runs, median)	$\leq$ 3 seconds including GraphQL request + scoring	**PLANNED**
TC-P007	Flipkart Playwright scraper latency	Measure `liveFlipkart()` execution time for one query	$\leq$ 45 seconds (includes browser launch, page load, DOM evaluation)	**PLANNED**
TC-P008	5 concurrent live search requests	Use `k6` to simulate 5 virtual users simultaneously sending POST `/api/search/live` with different queries	All 5 requests complete with HTTP 200; no 5xx errors; no deadlocks	**PLANNED**

Chapter 8

Security Test Cases

TC-ID	Test Name	Method	Expected Result	Status
TC-SEC001	Scrape endpoint — missing secret	POST `/api/scrape` with no `x-scrape-secret` header	HTTP 403; body: {"error":"Unautho no scrape triggered	PLANNED
TC-SEC002	Scrape endpoint — wrong secret	POST `/api/scrape` with header value `"wrongsecret"`	HTTP 403; body: {"error":"Unautho	PLANNED
TC-SEC003	SQL injection in search query	GET `/api/search?q=' OR '1'='1`	HTTP 200 with valid (possibly empty) results; no DB error; Prisma parameterisation prevents injection	PLANNED
TC-SEC004	No env variable leakage in response	Inspect all API response bodies for strings containing `DATABASE_URL`, `GROQ_API_KEY`, or `SCRAPE_SECRET`	No API response contains environment variable values	PLANNED

TC-ID	Test Name	Method	Expected Result	Status
TC-SEC005	No stack trace in error response	Send malformed JSON to `/api/ai/analyze`	HTTP 4xx/5xx; response body contains only a user-facing error message, not a Node.js stack trace	**PLANNED**
TC-SEC006	HTTPS enforced in production	Attempt HTTP request to the production URL (if deployed)	Redirected to HTTPS or connection refused; no plaintext communication	**PLANNED**

Chapter 9

Requirements Traceability Matrix

The table below maps each functional requirement from the SRS to the test cases that verify it.

FR-ID	Requirement Name	Covering Test Cases
FR-001	Live Search	TC-I001, TC-I002, TC-I003, TC-I004, TC-I005, TC-I006, TC-I007, TC-S003, TC-S004, TC-P001
FR-002	Product Identity Matching	TC-U001–TC-U040 (all unit tests cover the scoring engine); TC-I004, TC-I005 (verify correct product stored)
FR-003	Price Comparison Display	TC-S005, TC-S009, TC-I010, TC-I011, TC-I012
FR-004	AI Purchase Recommendation	TC-I008, TC-I009, TC-I015, TC-I016, TC-I017, TC-S006, TC-P004, TC-P005
FR-005	Price History Tracking	TC-I006, TC-S007, TC-I021, TC-I022, TC-P002
FR-006	Batch Scraping	TC-I018, TC-I019, TC-I020
FR-007	Database Text Search and Filtering	TC-I010, TC-I011, TC-I012, TC-I013, TC-I014, TC-S002, TC-S008
FR-008	On-Demand Price Refresh	TC-S011, TC-P006, TC-P007
FR-009	Administrative Dashboard	TC-S010

continued on next page...

FR-ID	Requirement Name	Covering Test Cases
FR-010	Dark Mode UI Theme	TC-S001, TC-S012
NFR-P01	Live search $\leq$ 60s	TC-P001
NFR-P02	Cached search $\leq$ 500ms	TC-P002
NFR-P03	Product page TTFB $\leq$ 2s	TC-P003
NFR-P04	AI latency $\leq$ 8s	TC-P004
NFR-P05	5 concurrent users	TC-P008
NFR-R01	Partial scrape failure graceful	TC-I009
NFR-S01	Scrape endpoint protection	TC-SEC001, TC-SEC002
NFR-S03	Input sanitisation	TC-SEC003
NFR-S04	No sensitive data in responses	TC-SEC004, TC-SEC005

Chapter 10

Test Summary

10.1 Test Case Count by Category

Category	Count	Automated?	Priority
Unit Tests (`matcher.ts`)	40	Yes (Jest)	Critical
Integration Tests (API routes)	23	Yes (Jest + Supertest)	High
System / E2E Tests	12	Manual / Playwright	High
Performance Tests	8	Manual / k6	Medium
Security Tests	6	Manual / OWASP ZAP	High
Total	**89**		

10.2 Known Limitations and Risks

1. **Amazon Bot Detection (Critical Risk):** Amazon.in returns HTTP 503 for programmatic requests. TC-I005 and TC-S003 will show Amazon results as null in all real test runs. This is a fundamental platform constraint, not a system defect.
2. **Playwright Fragility (Medium Risk):** Flipkart's DOM structure may change between evaluation periods, causing TC-P007 and related Flipkart tests to fail. The scraper would need a DOM selector update.
3. **Groq API Quota (Low Risk):** Integration tests that call the real Groq API will consume token quota. A mock should be used in CI environments.

4. **No Automated Test Suite Currently Configured (High Risk):** All 89 test cases are in *Planned* status. Configuring Jest and ts-jest is the highest-priority outstanding task before test execution.

10.3 Recommended Test Execution Order

1. Configure Jest + ts-jest in the project.
2. Run all unit tests (TC-U001–TC-U040) — no infrastructure required.
3. Start Docker PostgreSQL test instance.
4. Run integration tests (TC-I001–TC-I023) with mocked scrapers.
5. Start full application; run system tests manually (TC-S001–TC-S012).
6. Run performance tests (TC-P001–TC-P008) individually.
7. Run security tests (TC-SEC001–TC-SEC006).

Revision History

Version	Date	Author	Changes
1.0	April 2026	Adrij Bhadra	Initial release. 89 test cases across 5 categories. Full traceability matrix. Known limitations documented.